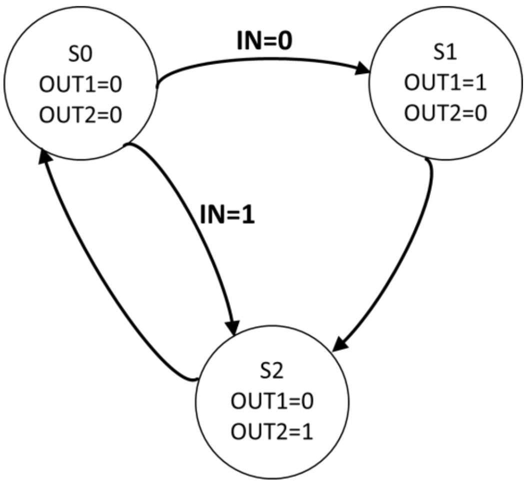
Tutorial 10 (SOLUTIONS)

NOTE THAT THERE ARE MANY OTHER POSSIBLE SOLUTIONS TOO…

SS1

1) State Transition Diagram



2) Next State Table

Current State	Inputs	Next State
S	**IN**	**S+**
S0	0	S1
S0	1	S2
S1	X	S2
S2	X	S0

By using two bits to represent the state, S_1S_0 :

State Assignment

	S_1	S_0
S0	0	0
S1	0	1
S2	1	0
S3	1	1

Current State		Inputs	Next State	
S_1	S_0	IN	S_1+	S_0+
0	0	0	0	1
0	0	1	1	0
0	1	X	1	0
1	0	X	0	0
1	1	X	0	0

The next state after S3 has been set to S0 instead of don't cares. This is a minimum-risk design choice because there is no external reset signal to this FSM.

A two bit counter can be designed using D Flip-flops for the state memory :

S_1S_0 \ IN	0	1
00	0	1
01	1	1
11	0	0
10	0	0

S_1S_0 \ IN	0	1
00	1	0
01	0	0
11	0	0
10	0	0

$D_1 = S_1^+ = \overline{S_1}\,IN + \overline{S_1}S_0$ $\qquad D_0 = S_0^+ = \overline{IN}\,\overline{S_1}\,\overline{S_0}$

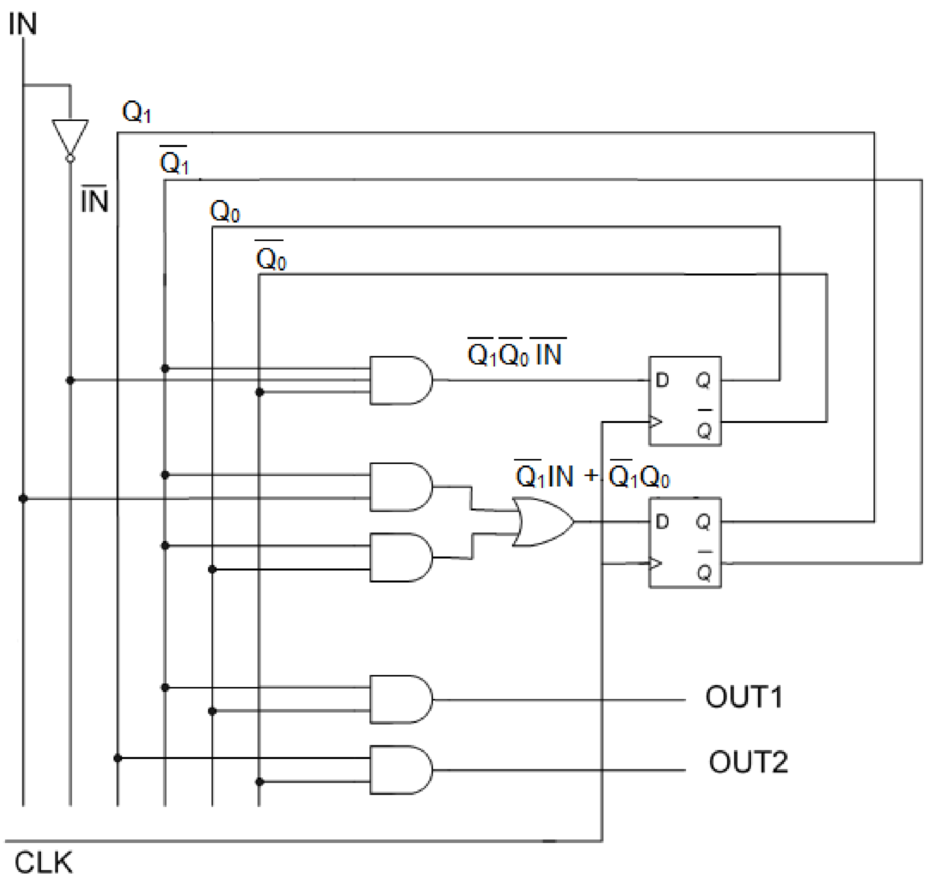
3) Output Logic Table

Current State	Outputs	
S	**OUT1**	**OUT2**
S0	0	0
S1	1	0
S2	0	1

Current State		Outputs	
S_1	**S_0**	**OUT1**	**OUT2**
0	0	0	0
0	1	1	0
1	0	0	1
1	1	X	X

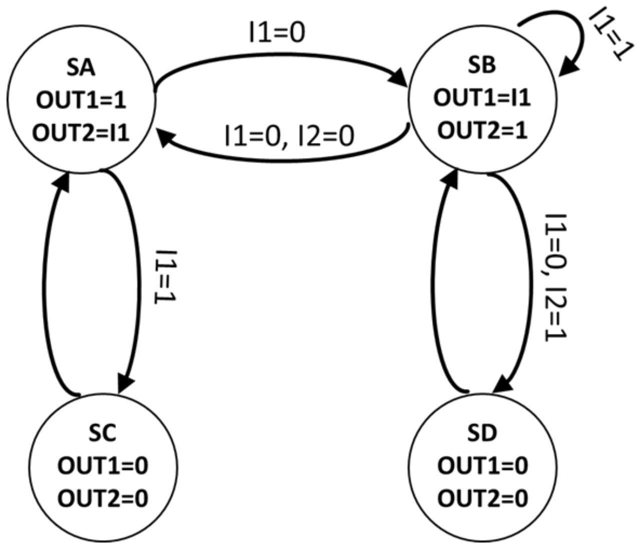
$$OUT_1 = S_1 = \overline{Q_1}Q_0, \quad OUT_2 = S_2 = Q_1\overline{Q_0}$$

4) Circuit Implementation



Notice that this is a Moore machine as the outputs (OUT1, OUT2) are a function of the current state (Q_1, Q_0) only.

1) State Transition Diagram



2) Next State Table

Current State	Inputs		Next State
S	**I1**	**I2**	**S+**
SA	0	X	SB
SA	1	X	SC
SB	0	0	SA
SB	0	1	SD
SB	1	X	SB
SC	X	X	SA
SD	X	X	SB

You can arrange the next state table like a KMAP to save yourself one step!

	I_1, I_2			
	00	01	11	10
SA	SB	SB	SC	SC
SB	SA	SD	SB	SB
SD	SB	SB	SB	SB
SC	SA	SA	SA	SA

A two bit counter can be designed using D Flip-flops for the state memory.

Applying the following State Assignment

	Q_1	**Q_0**
SA	0	0
SB	0	1
SC	1	0
SD	1	1

Q_1Q_0 \ I_1I_2	00	01	11	10
00	0	0	1	1
01	0	1	0	0
11	0	0	0	0
10	0	0	0	0

Q_1Q_0 \ I_1I_2	00	01	11	10
00	1	1	0	0
01	0	1	1	1
11	1	1	1	1
10	0	0	0	0

$$D_1 = Q_1^+ = \overline{Q_1}\,\overline{Q_0} I_1 + \overline{Q_1} Q_0 \overline{I_1} I_2$$

$$D_0 = Q_0^+ = \overline{Q_1}\,\overline{Q_0}\,\overline{I_1} + Q_1 Q_0$$
$$+ Q_0 I_2 + Q_0 I_1$$

3) Output Logic Table

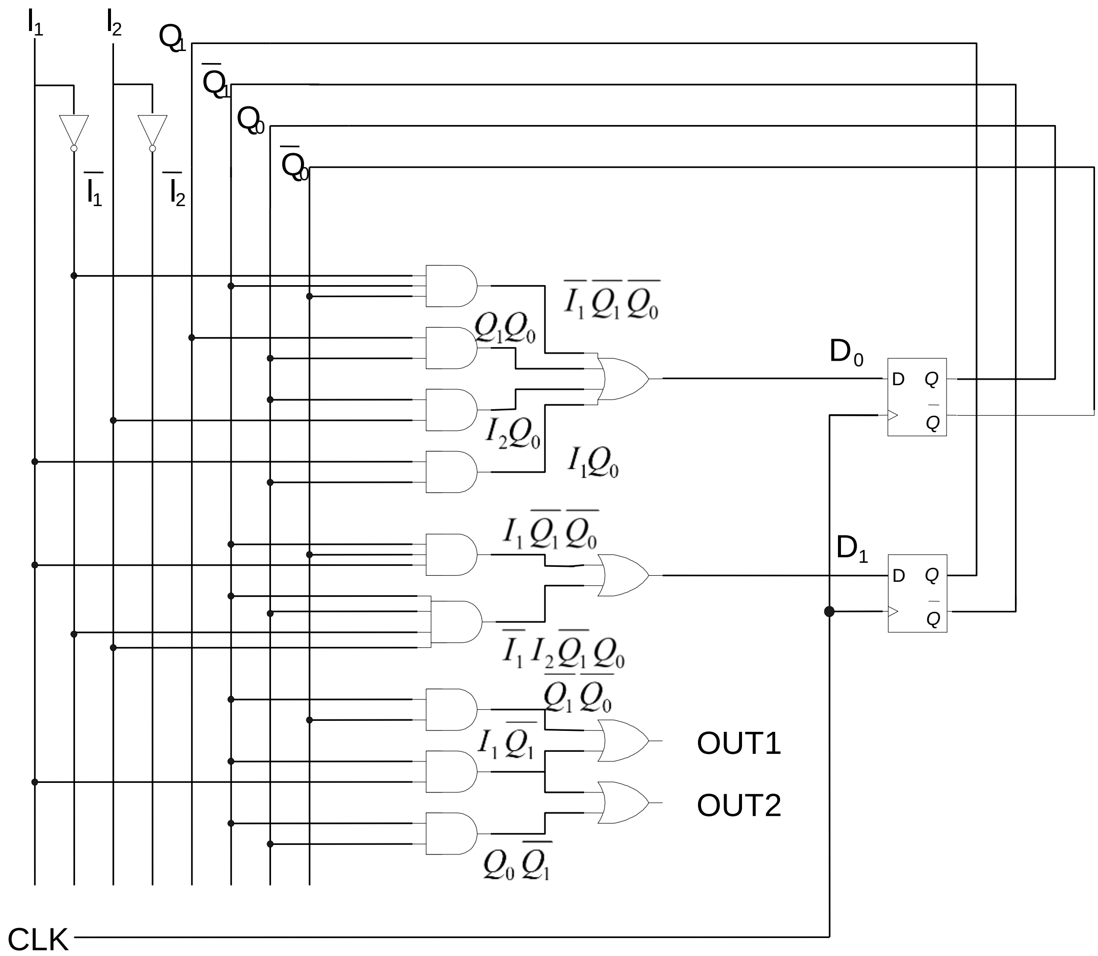
Current State	Outputs	
S	**OUT1**	**OUT2**
SA	1	I1
SB	I1	1
SC	0	0
SD	0	0

By inspection,

$$OUT1 = SA + SB.I_1$$
$$= \overline{Q_1}\,\overline{Q_0} + \overline{Q_1}Q_0 I_1 = \overline{Q_1}\,\overline{Q_0} + \overline{Q_1} I_1$$

$$OUT2 = SB + SA.I_1$$
$$= \overline{Q_1}\,\overline{Q_0} I_1 + \overline{Q_1} Q_0 = \overline{Q_1}\, Q_0 + \overline{Q_1} I_1$$

4) Circuit Implementation



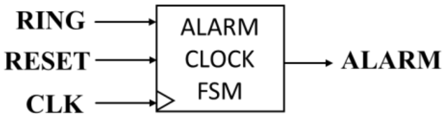
As the outputs, OUT1 and OUT2, are a function of both current state (Q_1, Q_0) as well as the current inputs (I1, I2), this is a Mealy machine.

Question 1

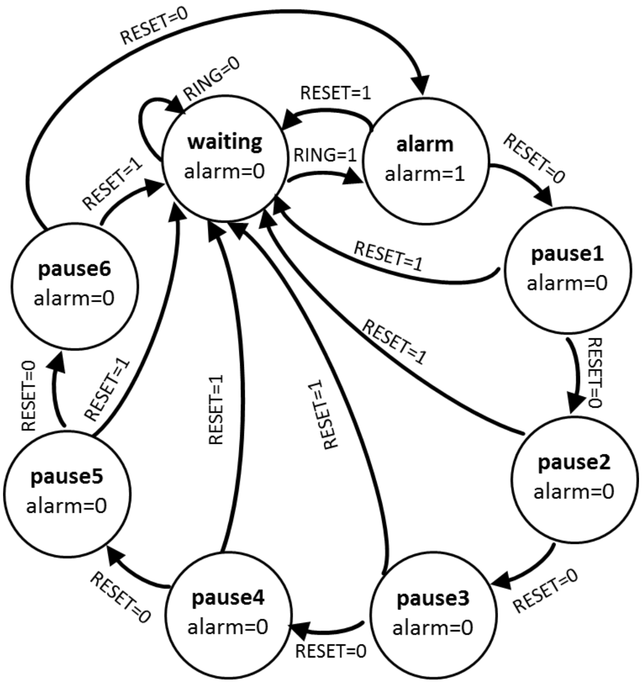
Selecting a clock period of 10 seconds, the snooze can be implemented by having 1 state of alarm (10 seconds) followed by 6 states of waiting (60 seconds).

Solution 1 :

Overall Block Diagram:

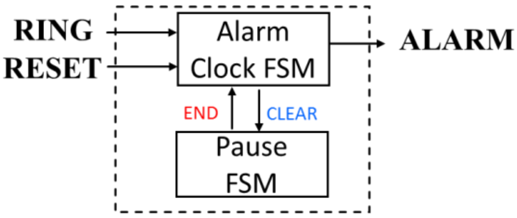


Alarm Clock FSM :

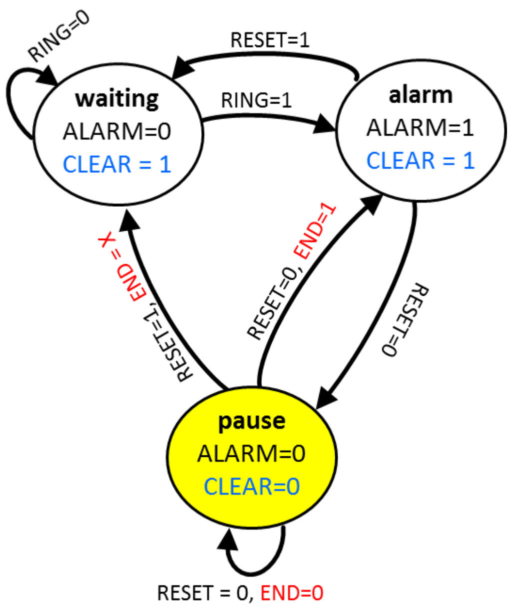


Solution 2 :

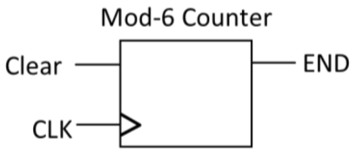
Modular FSM Block Diagram:



Alarm Clock FSM



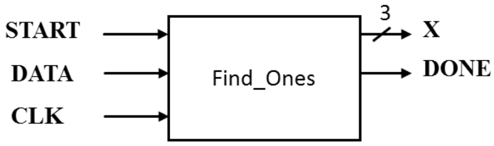
Pause FSM



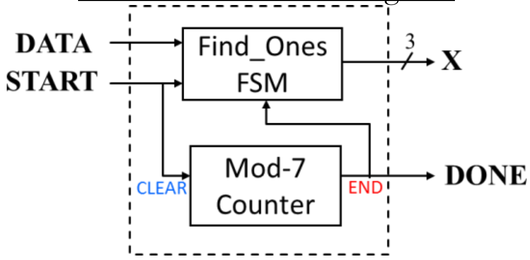
END= 1 *when* count = 5 (101)

Question 2

Overall Block Diagram :



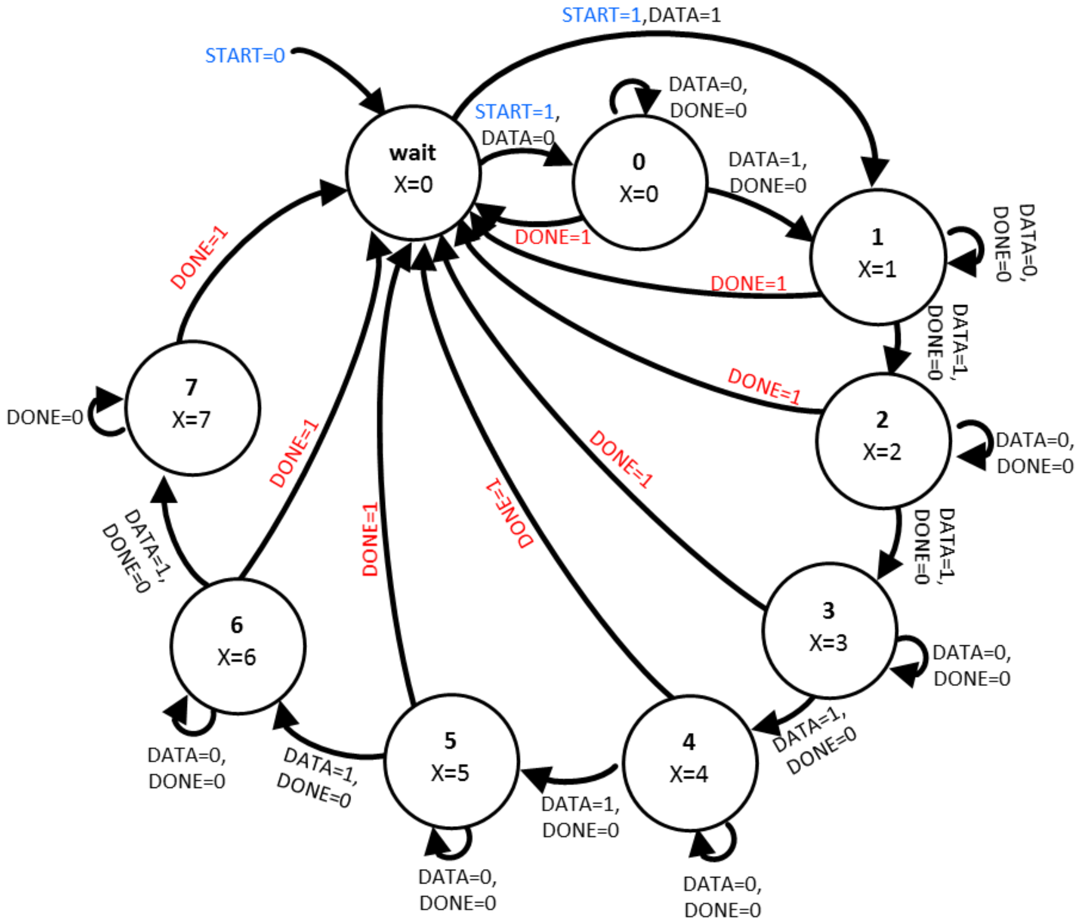
Modular FSM Block Diagram:



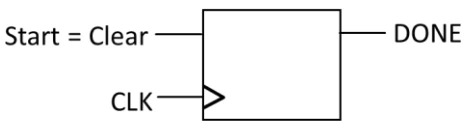
START = CLEAR & DONE = END

By reusing the START signal as the CLEAR signal, and the DONE signal as the END signal, we save two signals.

Find_Ones FSM :



Mod-7 Counter



DONE= 1 *when* count = 6 (110)

Question 3

Overall Block Diagram :

State Transition Diagram of PPL_COUNTER FSM :